Web API

Q1. What is ASP.NET Core Web API, and how does it differ from traditional ASP.NET MVC?

→

- ASP.NET Core Web API is a framework for building HTTP-based services that allow clients to access application data and functionality through endpoints.
- It is commonly used by Angular, React, mobile applications, and other backend services.
- ASP.NET Core MVC is commonly used to build web applications that return HTML pages using Razor Views. ASP.NET Core MVC can also return JSON and other HTTP responses.

2. Hinglish explanation

Web API ka use backend services aur endpoints banane ke liye hota hai. Angular frontend API ko call karta hai, API database se data fetch ya update karti hai, aur response JSON format mein return kar sakti hai.

MVC mein hum generally Razor Views ke through complete HTML pages return karte hain.

3. Short code example

Web API controller:

```
[ApiController]
[Route("api/[controller]")]
public class LoansController : ControllerBase
{
   [HttpGet]
   public IActionResult GetLoans()
   {
      var loans = new[] { "Home Loan", "Car Loan" };
      return Ok(loans);
   }
}
```

Example endpoint:

GET /api/Loans

Response:

```
[
  "Home Loan",
  "Car Loan"
]
```

MVC controller example:

```
public class HomeController : Controller
{
   public IActionResult Index()
   {
      return View();
   }
}
```

Here, View() returns a Razor View, typically rendered as HTML.

4. Quick comparison

ASP.NET Core Web API	**ASP.NET Core MVC**
Commonly returns JSON or other API responses	Commonly returns HTML views
Uses ControllerBase for API controllers	Uses Controller for MVC controllers with view support
Commonly consumed by Angular, React, or mobile apps	Commonly used for server-rendered web pages
Uses HTTP status codes to communicate API results	Can return views, redirects, JSON, and status codes

Interview tip: Web API and MVC are not completely separate technologies. ASP.NET Core MVC infrastructure supports both API controllers and view-based controllers.

Q2. What is a RESTful API? Explain its principles.

→

A RESTful API is an API that follows REST (Representational State Transfer) architectural principles to allow clients and servers to communicate over HTTP.

Its main principles are:

1. **Client–Server:** Client and server responsibilities are separated.
2. **Stateless:** Each request contains the information needed to process it; the server does not rely on stored client-session state between requests.
3. **Cacheable:** Responses can indicate whether they may be cached.
4. **Uniform Interface**: Resources are accessed through consistent URLs and HTTP methods.
5. **Layered System:** The client may communicate through intermediaries such as proxies or gateways.

2. Hinglish Explanation

RESTful API ek standard approach hai jisme frontend aur backend HTTP ke through communicate karte hain.

For example, tumhare Loan Application project mein Angular frontend API ko call karega, aur API loan details database se fetch karke JSON response return karegi.

- GET — data fetch karna.
- POST — naya data create karna.
- PUT — resource ko update ya replace karna.
- DELETE — resource delete karna.

3. Short Code Example

```
[ApiController]

[Route("api/[controller]")]

public class LoansController : ControllerBase

{

   [HttpGet("{id}")]

   public IActionResult GetLoan(int id)

   {

      return Ok(new { LoanId = id, Amount = 50000 });

   }


   [HttpPost]

   public IActionResult CreateLoan()

   {

      return StatusCode(201, "Loan created");

   }

}
```

4. Interview Tip

REST is an architectural style, not a protocol. HTTP is the protocol commonly used to implement RESTful APIs.

Q3. What are the different HTTP methods supported by ASP.NET Core Web API, and what are their purposes?

→

HTTP methods define what action a client wants to perform on a resource through an API.

The most commonly used methods are:

- GET: Retrieve data.
- POST: Create a new resource or submit data for processing.
- PUT: Replace or fully update a resource.
- PATCH: Partially update a resource.
- DELETE: Delete a resource.
- HEAD: Retrieve response headers without the response body.
- OPTIONS: Discover supported communication options for a resource.

2. Hinglish Explanation

Jab Angular frontend backend API ko request bhejta hai, toh HTTP method batata hai ki data ke saath kya operation perform karna hai.

For example, Loan Application project mein:

- GET — loan details dekhna.
- POST — naya loan create karna.
- PUT — loan ki complete details update karna.
- PATCH — sirf loan status update karna.
- DELETE — loan record delete karna.

3. Short Code Example

```
[ApiController]
[Route("api/[controller]")]
public class LoansController : ControllerBase
{
   [HttpGet("{id}")]
   public IActionResult GetLoan(int id)
   {
      return Ok(new { LoanId = id, Amount = 50000 });
   }
```

```
    [HttpPost]
    public IActionResult CreateLoan()
    {
        return StatusCode(201, "Loan created");
    }

    [HttpPut("{id}")]
    public IActionResult UpdateLoan(int id)
    {
        return Ok($"Loan {id} updated");
    }

    [HttpDelete("{id}")]
    public IActionResult DeleteLoan(int id)
    {
        return NoContent();
    }
}
```

4. Quick Comparison

Method	**Purpose**	**Typical success status**
GET	Retrieve data	200 OK
POST	Create/process data	201 Created or 200 OK
PUT	Replace a resource	200 OK or 204 No Content
PATCH	Partially update	200 OK or 204 No Content
DELETE	Delete a resource	204 No Content

Interview tip: GET should not modify resource state. PUT and DELETE are generally idempotent, meaning repeating the same request has the same intended effect. POST is not generally idempotent.

Q4. What is Model Binding in ASP.NET Core Web API?

→

Model Binding is the process of mapping incoming HTTP request data to action method parameters or C# model properties automatically.

ASP.NET Core Web API can bind data from:

- [FromRoute] — URL route parameters.
- [FromQuery] — Query string parameters.
- [FromBody] — Request body, commonly JSON.
- [FromHeader] — HTTP headers.
- [FromForm] — Form data.

2. Hinglish Explanation

Model Binding ka matlab hai HTTP request se aane wale data ko automatically C# variables ya model objects mein convert karna.

For example, Angular se loan ID, query parameters ya JSON data API ko bhejte hain. ASP.NET Core us data ko action method ke parameters mein bind kar sakta hai.

3. Short Code Example

```
[HttpGet("{id}")]
public IActionResult GetLoan(
   [FromRoute] int id,
   [FromQuery] string status)
{
   return Ok(new { LoanId = id, Status = status });
}


[HttpPost]
public IActionResult CreateLoan([FromBody] LoanDto loan)
{
   return Ok(loan);
}
```

Example requests:

- GET /api/loans/10?status=Active
- POST /api/loans with JSON body:

```
{
  "loanAmount": 50000,
  "loanType": "Home"
}
```

In the GET request, id comes from the route and status comes from the query string. In the POST request, the JSON body is bound to LoanDto.

4. Interview Tip

With [ApiController], ASP.NET Core infers binding sources for many parameter types and automatically returns HTTP 400 for model validation errors. You can still use explicit attributes when you want to make the source clear.

Q5. How do you handle routing in ASP.NET Core Web API? Explain Attribute Routing and Convention-Based Routing.

→

Routing determines which controller action should handle an incoming HTTP request based on its URL and HTTP method.

ASP.NET Core supports two routing approaches:

- Attribute Routing: Routes are defined directly on controllers and action methods using attributes such as [Route], [HttpGet], and [HttpPost].
- Convention-Based Routing: Routes are defined centrally using a routing convention, commonly configured with MapControllerRoute().

2. Hinglish Explanation

Routing decide karti hai ki incoming API request kis controller aur action method par jayegi.

- Attribute Routing: Controller ya method ke upar route define karte hain.
- Convention-Based Routing: Ek common route pattern define karte hain, jiske according controller aur action select hote hain.

3. Short Code Example

Attribute Routing — commonly used in Web APIs:

```
[ApiController]
[Route("api/[controller]")]
public class LoansController : ControllerBase
{
    [HttpGet("{id}")]
```

```
    public IActionResult GetLoan(int id)
    {
        return Ok(new { LoanId = id });
    }
}
```

Request: GET /api/loans/10

Here, [controller] becomes loans, and {id} receives the value 10.

Convention-Based Routing:

```
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

Example URL: /Home/Index/10

This pattern maps the request to the HomeController, Index action, with an optional id of 10.

4. Quick Comparison

Attribute Routing	**Convention-Based Routing**
Defined using route attributes	Defined using central route patterns
Provides precise URL control	Provides consistent URL patterns
Common in Web APIs	Common in MVC applications

Interview tip: Attribute routing is commonly preferred for REST APIs because it makes endpoint URLs and HTTP methods explicit.

Q6. What are DTOs (Data Transfer Objects), and why do we use them in Web API?

→

A DTO (Data Transfer Object) is a class used to transfer specific data between the client and the application.

We use DTOs to:

- Expose only the required properties.
- Prevent clients from directly controlling sensitive entity properties.
- Separate API request/response structures from database entities.
- Apply validation rules to incoming data.

2. Hinglish Explanation

DTO ek separate class hoti hai jiske through frontend aur backend ke beech data transfer karte hain.

Maan lo tumhare Loan Application project mein LoanAccount entity mein 15 properties hain, lekin loan create karte waqt frontend ko sirf 3 properties bhejni hain. Toh hum CreateLoanDto banayenge.

Isse frontend directly database entity ko control nahi karega.

3. Short Code Example

Entity:

```
public class LoanAccount
{
    public int LoanAccountId { get; set; }
    public decimal LoanAmount { get; set; }
    public string LoanStatus { get; set; } = "";
    public int CustomerId { get; set; }
}
```

DTO:

```
public class CreateLoanDto
{
    public decimal LoanAmount { get; set; }
    public int CustomerId { get; set; }
}
```

Controller:

```
[HttpPost]
public IActionResult CreateLoan(CreateLoanDto dto)
{
    var loan = new LoanAccount
    {
        LoanAmount = dto.LoanAmount,
        CustomerId = dto.CustomerId,
        LoanStatus = "Pending"
    };

    // Save loan using your service or DbContext
    return Ok("Loan created");
```

}

Here, the client cannot directly set LoanStatus through this DTO. The backend controls its initial value.

4. Quick Comparison

Entity	**DTO**
Represents application/database data	Represents data transferred through the API
May contain many properties	Contains only properties needed for a particular operation
Used by application and persistence logic	Used for API requests and responses

Interview tip: It's common to create separate DTOs for creating, updating, and reading data—for example, CreateLoanDto, UpdateLoanDto, and LoanResponseDto.

Q7. What is ActionResult<T> in ASP.NET Core Web API, and why do we use it?

→

ActionResult<T> allows a Web API action to return either a strongly typed response object (T) or an HTTP action result such as NotFound() or BadRequest().

It is useful when an API needs to return data on success and appropriate HTTP status codes when something goes wrong.

2. Hinglish Explanation

ActionResult<T> ka use tab karte hain jab API se successful hone par data return karna ho, aur failure hone par 404 Not Found ya 400 Bad Request jaisa status code.

For example, Loan Application API mein loan milne par loan details return karenge. Agar loan nahi mila, toh NotFound() return karenge.

3. Short Code Example

```
[HttpGet("{id}")]

public async Task<ActionResult<LoanDto>> GetLoan(int id)

{

   var loan = await service.GetLoanById(id);


   if (loan == null)

      return NotFound();
```

```
    return Ok(loan);
}
```

How it works:

- Loan exists → 200 OK with loan data.
- Loan doesn't exist → 404 Not Found.

Here, LoanDto defines the expected response data type.

4. Quick Comparison

IActionResult	**ActionResult<T>**
Can return different action results	Can return typed data or action results
Doesn't declare a specific response data type	Declares an expected response data type
Useful for varied response types	Useful for APIs returning a known data model

Interview tip: Use ActionResult<T> when an endpoint normally returns a specific DTO but may also return error results.

Q8. How do you return custom HTTP status codes from ASP.NET Core Web API?

→

In ASP.NET Core Web API, we use HTTP status codes to communicate the result of a request to the client.

Common methods include:

- Ok() — 200: Request successful.
- CreatedAtAction() — 201: Resource created.
- BadRequest() — 400: Invalid request.
- Unauthorized() — 401: Authentication required or invalid.
- Forbid() — 403: Access denied.
- NotFound() — 404: Resource not found.
- StatusCode() — Returns a specified status code.

2. Hinglish Explanation

Jab Angular frontend API call karta hai, toh backend status code return karta hai, jisse frontend ko pata chalta hai ki request successful thi ya error aaya.

For example, loan create hone par 201 Created, loan na milne par 404 Not Found, aur invalid input par 400 Bad Request return kar sakte hain.

3. Short Code Example

```
[HttpPost]
public IActionResult CreateLoan(CreateLoanDto dto)
{
   if (dto.LoanAmount <= 0)
      return BadRequest("Loan amount must be greater than zero.");

   // Save the loan here

   return StatusCode(201, new
   {
      Message = "Loan created successfully"
   });
}
```

You can also return a custom status code with a message:

```
return StatusCode(409, "Loan already exists.");
```

Here, 409 Conflict indicates that the request conflicts with the current state of the resource.

4. Quick Comparison

Method	**Status code**	**Meaning**
Ok()	200	Success
CreatedAtAction()	201	Resource created
BadRequest()	400	Invalid request
Unauthorized()	401	Not authenticated
Forbid()	403	Not authorized
NotFound()	404	Resource not found
StatusCode(409, ...)	409	Conflict

Interview tip: Use the status code that accurately represents the outcome. Prefer CreatedAtAction() when creating a resource and you can provide a route to retrieve it.

Q9. What is Content Negotiation in ASP.NET Core Web API?

→

- Content Negotiation is the process by which an API determines the response format based on the client's request headers, especially the Accept header.
- For example, a client may request JSON or XML. The API can return a supported format based on the request and its configured formatters.

2. Hinglish Explanation

Content Negotiation ka matlab hai client aur server ke beech response format decide karna.

Client Accept header mein batata hai ki use kis format mein response chahiye, jaise **JSON ya XML**. API supported format ke according response return kar sakti hai.

3. Short Code Example

Request:

```
GET /api/loans

Accept: application/json
```

Controller:

```
[HttpGet]

public IActionResult GetLoans()

{

   var loans = new[]

   {

      new { LoanId = 1, Amount = 50000 }

   };


   return Ok(loans);

}
```

By default, ASP.NET Core Web API commonly returns JSON.

If you want to support XML, configure an XML formatter:

```
builder.Services.AddControllers()

   .AddXmlSerializerFormatters();
```

A client can then request XML:

```
GET /api/loans

Accept: application/xml
```

The server can return XML when the formatter is configured and the requested format is supported.

4. Quick Comparison

Accept header	Requested response
application/json	JSON
application/xml	XML, if supported
/	Any acceptable format

Interview tip: The Accept header specifies the response format the client can accept. The Content-Type header specifies the format of the request body or response body being sent.

Q10. How do you implement pagination in ASP.NET Core Web API?

→

Pagination is a technique for returning data in smaller pages instead of retrieving all records at once.

It improves API performance, reduces unnecessary data transfer, and makes large datasets easier to display.

Common parameters are:

- pageNumber: The page to retrieve.
- pageSize: The number of records per page.

2. Hinglish Explanation

Suppose tumhare Loan Application database mein 10,000 loan records hain. Saare records ek saath frontend par bhejne ke bajaye, hum har request mein sirf 10 ya 20 records bhejte hain.

- pageNumber batata hai kaunsa page chahiye.
- pageSize batata hai ek page mein kitne records chahiye.

3. Short Code Example

```
[HttpGet]
public async Task<IActionResult> GetLoans(
    int pageNumber = 1,
    int pageSize = 10)
{
    if (pageNumber < 1 || pageSize < 1 || pageSize > 100)
        return BadRequest("Invalid pagination parameters.");
```

```
    var loans = await db.LoanAccounts
        .AsNoTracking()
        .OrderBy(l => l.LoanAccountId)
        .Skip((pageNumber - 1) * pageSize)
        .Take(pageSize)
        .ToListAsync();

    return Ok(loans);
}
```

Example request:

GET /api/loans?pageNumber=2&pageSize=10

How it works:

- Page 1 skips 0 records and takes 10.
- Page 2 skips 10 records and takes the next 10.
- Page 3 skips 20 records and takes the next 10.

The OrderBy() ensures consistent ordering for pagination when the ordering key is unique.

4. Interview Tip

For production APIs, consider returning pagination metadata such as total record count, current page, and total pages. For very large datasets, keyset pagination can be more efficient than Skip() and Take().

Q11. What is JWT Authentication, and how does it work in ASP.NET Core Web API?

→

“JWT stands for JSON Web Token. It is a compact token format commonly used to authenticate users and securely transmit claims between a client and a server.

When a user logs in with valid credentials, the server generates a signed JWT and returns it to the client. The client sends this token in the Authorization header with subsequent API requests. The API validates the token's signature, expiration, and other configured requirements before allowing access to protected endpoints.

JWT authentication is commonly used in Web APIs because it supports stateless authentication.”

2. Hinglish Explanation

Jab user login karta hai, server credentials verify karke JWT token generate karta hai. Frontend token receive karta hai aur protected API ko call karte waqt Bearer token bhejta hai.

Server token validate karta hai. Agar token valid hai aur user authorized hai, toh API request process karti hai.

3. Short Example

Client request:

GET /api/loans

Authorization: Bearer <your-jwt-token>

Protect an endpoint in ASP.NET Core:

```
[Authorize]

[HttpGet]

public IActionResult GetLoans()

{

   return Ok("Authorized access");

}
```

JWT bearer authentication must also be configured in Program.cs for [Authorize] to validate the token.

4. Interview Tip

Remember the flow:

Login → Validate credentials → Generate JWT → Send token → Validate token → Access API

A JWT is typically signed, not encrypted by default. Avoid putting passwords or other secrets in its payload.

Q12. How do you implement role-based authorization in ASP.NET Core Web API?

→

- “Role-based authorization is used to restrict access to API endpoints based on a user's role, such as Admin, Manager, or User.
- In ASP.NET Core Web API, we implement it using the [Authorize] attribute with the Roles parameter. First, the user authenticates, and their role is included in the authentication claims, such as a JWT token. When the user accesses a protected endpoint, ASP.NET Core checks whether their role is permitted to access that endpoint.
- This helps ensure that users can access only the resources and operations allowed for their roles.”

2. Hinglish Explanation

Maan lo hamare Loan Application project mein do roles hain: Admin aur User.

- Admin: Penalty apply kar sakta hai.
- User: Apni loan details aur foreclosure requests dekh sakta hai.

Hum [Authorize(Roles = "Admin")] use karke sirf Admin ko endpoint access karne denge. Agar User us endpoint ko access karega, toh usse access deny ho jayega.

3. Short Code Example

Admin-only endpoint:

```
[Authorize(Roles = "Admin")]

[HttpPost("ApplyPenalty")]

public IActionResult ApplyPenalty()

{

   return Ok("Penalty applied successfully");

}
```

User-only endpoint:

```
[Authorize(Roles = "User")]

[HttpGet("MyLoans")]

public IActionResult GetMyLoans()

{

   return Ok("User loan details");

}
```

Make sure authentication is configured, the JWT contains the appropriate role claim, and the authentication middleware runs before authorization middleware.

4. Interview Tip

Remember the difference:

- [Authorize] — authenticated users can access the endpoint, subject to any other configured policies.
- [Authorize(Roles = "Admin")] — only users whose role matches Admin can access it.
- [AllowAnonymous] — allows access without authentication for that endpoint.

Q13. What is CORS, and how do you configure it in ASP.NET Core Web API?

→

- "CORS stands for Cross-Origin Resource Sharing. It is a browser security mechanism that controls whether a web application running on one origin can access resources from a different origin.
- For example, if my Angular frontend runs on http://localhost:4200 and my ASP.NET Core Web API runs on https://localhost:7008, they have different origins.
- In ASP.NET Core, I configure CORS by defining a policy using AddCors() and applying it using UseCors(). This allows the frontend to access the API from the specified origin."

2. Hinglish Explanation

CORS ka issue tab aata hai jab frontend aur backend different origins par run kar rahe hote hain.

Tumhare Loan Application project mein Angular frontend aur .NET Web API alag ports par run kar sakte hain. Browser request ko block kar sakta hai agar backend ne frontend ke origin ko allow nahi kiya hai.

Isliye hum backend mein CORS policy configure karte hain.

3. Short Code Example

Program.cs

```
builder.Services.AddCors(options =>
{
   options.AddPolicy("AngularPolicy", policy =>
   {
      policy.WithOrigins("http://localhost:4200")
            .AllowAnyHeader()
            .AllowAnyMethod();
   });
});
```

Configure the middleware before mapping controllers:

```
app.UseCors("AngularPolicy");


app.UseAuthentication();
app.UseAuthorization();


app.MapControllers();
```

This policy allows requests from the specified Angular origin with any HTTP method and header. In production, specify only trusted origins.

4. Interview Tip

Remember these three methods:

- WithOrigins() — specifies allowed origins.
- AllowAnyHeader() — allows any request header.
- AllowAnyMethod() — allows any HTTP method.

Q14. How do you secure an API using Bearer Tokens in ASP.NET Core Web API?

→

- "Bearer token authentication is a mechanism used to secure API endpoints by requiring clients to send an access token with each request.
- In ASP.NET Core Web API, the client sends the token in the Authorization header using the format Bearer <token>. The API uses JWT Bearer authentication middleware to validate the token's signature, expiration, issuer, and audience, depending on the configured validation parameters.
- If the token is valid, the request proceeds to authorization. Otherwise, the API typically returns HTTP 401 Unauthorized. We can use the [Authorize] attribute to protect endpoints."

2. Hinglish Explanation

Bearer token ka matlab hai ki client apne request ke saath token bhejta hai.

Flow:

1. User login karta hai.
2. Backend JWT token generate karta hai.
3. Frontend token ko request ke Authorization header mein bhejta hai.
4. Backend token validate karta hai.
5. Valid token hone par request aage process hoti hai.

3. Short Code Example

Angular/client request header:

Authorization: Bearer eyJhbGciOiJIUzI1NiIs...

ASP.NET Core endpoint:

```
[Authorize]

[HttpGet("profile")]

public IActionResult GetProfile()

{

   return Ok("Access granted");

}
```

In Program.cs, configure JWT Bearer authentication and its validation parameters, then register the middleware in the correct order:

```
app.UseAuthentication();

app.UseAuthorization();

app.MapControllers();
```

4. Interview Tip

Remember the difference:

- 401 Unauthorized: Authentication is missing or invalid.
- 403 Forbidden: The user is authenticated but lacks the required permission.

Also, always use HTTPS and never expose access tokens in URLs or logs.

Q15. How do you handle exceptions globally in ASP.NET Core Web API?

→

- “Global exception handling is used to handle unexpected errors in one centralized place instead of writing try-catch blocks in every controller action.
- In ASP.NET Core Web API, we can implement global exception handling using custom exception-handling middleware or the built-in UseExceptionHandler() middleware.
- When an exception occurs, the middleware catches it, logs the error, and returns an appropriate HTTP status code with a consistent error response. This improves code maintainability and prevents internal exception details from being exposed to clients.”

2. Hinglish Explanation

Maan lo Loan Application API mein database connection fail ho gaya. Agar har controller mein alag-alag try-catch likhenge, toh code repetitive ho jayega.

Isliye hum global exception handling middleware use karte hain. Ye application mein exceptions ko centrally handle karta hai aur client ko appropriate error response deta hai.

3. Short Code Example

Program.cs

```
app.UseExceptionHandler("/error");
```

Create an error endpoint:

```
[ApiExplorerSettings(IgnoreApi = true)]
[Route("/error")]
public IActionResult HandleError()
{
   return Problem(
      statusCode: StatusCodes.Status500InternalServerError,
      title: "An unexpected error occurred."
   );
}
```

This example uses the built-in exception handler and a dedicated endpoint. In a production API, also configure logging and ensure sensitive exception details are not returned to clients.

4. Interview Tip

- Global middleware: handles exceptions centrally.
- Logging: records diagnostic details for developers.
- HTTP 500: indicates an unexpected server-side error.
- Consistent response: makes errors easier for API clients to handle.

Q16. What are API Filters in ASP.NET Core Web API?

→

- “Filters in ASP.NET Core Web API allow us to execute code at specific stages of the request processing pipeline, before or after certain actions are executed.
- They are used for cross-cutting concerns such as logging, authorization, validation, and exception handling. ASP.NET Core provides several types of filters, including Authorization Filters, Resource Filters, Action Filters, Exception Filters, and Result Filters.
- For example, an Action Filter can execute code before and after a controller action runs. Filters help us reduce code duplication and keep controllers clean.”

2. Hinglish Explanation

Filters ka use tab hota hai jab humein controller actions ke execute hone se pehle ya baad mein koi common task perform karna ho.

Example: Agar humein multiple API actions ka execution time log karna hai, toh har action mein logging code likhne ke bajay hum Action Filter bana sakte hain.

3. Short Code Example

```
public class LogActionFilter : IActionFilter
{
   public void OnActionExecuting(ActionExecutingContext context)
   {
      Console.WriteLine("Action is starting");
   }

   public void OnActionExecuted(ActionExecutedContext context)
   {
      Console.WriteLine("Action has completed");
   }
}
```

Apply the filter to a controller:

```
[ServiceFilter(typeof(LogActionFilter))]
[ApiController]
[Route("api/[controller]")]
public class LoansController : ControllerBase
{
   [HttpGet]
```

```
    public IActionResult GetLoans()
    {
        return Ok("Loan list");
    }
}
```

Register the filter in Program.cs:

builder.Services.AddScoped<LogActionFilter>();

4. Interview Tip

Remember the five main filter types:

- Authorization Filter: handles authorization checks.
- Resource Filter: runs around most of the remaining pipeline.
- Action Filter: runs before and after an action method.
- Exception Filter: handles certain unhandled exceptions during MVC action execution.
- Result Filter: runs before and after action results execute.

Q17. How do you implement API Versioning in ASP.NET Core Web API?

→

- "API versioning allows us to maintain multiple versions of an API so that we can introduce changes without breaking existing clients.
- In ASP.NET Core Web API, we can implement versioning using URL path versioning, query string versioning, or header-based versioning. For example, /api/v1/loans can represent version 1, while /api/v2/loans represents version 2.
- This approach allows existing clients to continue using the older version while new clients use the updated API. It improves backward compatibility and makes API maintenance easier."

2. Hinglish Explanation

Maan lo tumhare Loan Application project mein GetLoans API pehle sirf loan ID aur amount return karti thi. Ab tumhe customer details bhi return karni hain.

Agar existing API ko directly change kar diya, toh purana frontend break ho sakta hai. Isliye hum v1 aur v2 maintain kar sakte hain.

- api/v1/loans — purana response.
- api/v2/loans — updated response.

3. Short Code Example

Version 1 controller:

```
[ApiController]
[Route("api/v1/loans")]
public class LoansV1Controller : ControllerBase
{
   [HttpGet]
   public IActionResult GetLoans()
   {
      return Ok(new { LoanId = 1, Amount = 50000 });
   }
}
```

Version 2 controller:

```
[ApiController]
[Route("api/v2/loans")]
public class LoansV2Controller : ControllerBase
{
   [HttpGet]
   public IActionResult GetLoans()
   {
      return Ok(new { LoanId = 1, Amount = 50000, CustomerName = "Rahul" });
   }
}
```

This demonstrates URL-based versioning using separate routes. For larger APIs, you can use a dedicated API versioning library to manage versions more systematically.

4. Interview Tip

Remember the three common approaches:

- URL-based: /api/v1/loans
- Query string: /api/loans?api-version=1.0
- Header-based: version specified in a request header.

Q18. How do you implement caching in ASP.NET Core Web API?

→

- “Caching is a technique used to store frequently accessed data temporarily so that it can be retrieved faster without repeatedly executing expensive operations or querying the database.
- In ASP.NET Core Web API, we can implement caching using in-memory caching, distributed caching, or response caching. In-memory caching stores data in the application's memory, while distributed caching stores data in a shared cache accessible to multiple application instances.
- For example, I can use IMemoryCache to cache frequently accessed loan data. This improves response time and reduces database load. Cached data should have an appropriate expiration policy to prevent stale results.”

2. Hinglish Explanation

Maan lo Loan Application API mein loan interest rates baar-baar fetch ho rahe hain. Har request par database query chalane ke bajay hum data ko cache mein store kar sakte hain.

- In-memory caching: Data application ki memory mein store hota hai.
- Distributed caching: Data shared cache, jaise Redis, mein store hota hai.
- Response caching: HTTP response ko caching rules ke according cache kar sakte hain.

3. Short Code Example

Register memory cache in Program.cs:

```
builder.Services.AddMemoryCache();
```

Use it in a controller:

```
private readonly IMemoryCache _cache;


public LoansController(IMemoryCache cache)
{
   _cache = cache;
}


[HttpGet("interest-rates")]
public IActionResult GetInterestRates()
{
   var rates = _cache.GetOrCreate("InterestRates", entry =>
   {
```

```
        entry.AbsoluteExpirationRelativeToNow = TimeSpan.FromMinutes(10);
        return new[] { 8.5, 9.0, 9.5 };
    });

    return Ok(rates);
}
```

Here, the interest rates are cached for up to 10 minutes. In a real application, the values could come from a database or another service.

4. Interview Tip

Remember these differences:

- IMemoryCache — suitable for a single application instance or local caching.
- IDistributedCache — suitable when multiple instances need shared cached data.
- Response caching — caches HTTP responses according to caching rules.

Q19. How do you consume another API using HttpClient in ASP.NET Core?

→

- "In ASP.NET Core, we use HttpClient to send HTTP requests to external APIs and consume their responses.
- The recommended approach is to register a typed or named HttpClient using IHttpClientFactory. This helps manage the HttpClient lifecycle and avoids common issues associated with improper connection handling.
- We can use methods such as GetAsync(), PostAsync(), and SendAsync() to communicate with another API. We then process the response, deserialize JSON data when required, and handle errors appropriately."

2. Hinglish Explanation

Maan lo tumhare Loan Application project ko kisi external API se CIBIL score fetch karna hai.

Tum HttpClient ke through external API ko request bhejoge, response receive karoge aur JSON data ko C# object mein convert kar sakte ho.

Flow: Your API → External API request → Response → Deserialize JSON → Use the data.

3. Short Code Example

Step 1: Register HttpClient in Program.cs

```
builder.Services.AddHttpClient();
```

Step 2: Consume an external API

```
public class LoanService
{
    private readonly HttpClient _httpClient;

    public LoanService(IHttpClientFactory factory)
    {
        _httpClient = factory.CreateClient();
    }

    public async Task<string> GetDataAsync()
    {
        var response = await _httpClient.GetAsync(
            "https://jsonplaceholder.typicode.com/posts/1");

        response.EnsureSuccessStatusCode();

        return await response.Content.ReadAsStringAsync();
    }
}
```

Here, IHttpClientFactory creates the client, GetAsync() sends the request, and EnsureSuccessStatusCode() throws an exception if the response indicates an unsuccessful HTTP status.

4. Interview Tip

Remember these three points:

- GetAsync() — sends a GET request.
- PostAsync() — sends a POST request.
- IHttpClientFactory — recommended for managing HTTP client creation.

For production applications, also consider timeouts, cancellation tokens, logging, and retry policies where appropriate.

Q20. How do you test Web APIs in ASP.NET Core?

→

- "In ASP.NET Core, we can test Web APIs using tools such as Postman, Swagger, and automated testing frameworks.
- Postman allows us to send HTTP requests, provide headers and request bodies, and verify response status codes and data. Swagger provides an interactive interface to explore and test API endpoints.
- For automated testing, we can use xUnit or NUnit for unit testing and Microsoft.AspNetCore.Mvc.Testing with WebApplicationFactory for integration testing. Unit tests verify individual components, while integration tests verify how multiple components work together."

2. Hinglish Explanation

Web API testing ka matlab hai check karna ki API expected result de rahi hai ya nahi.

- Postman: API ko manually test karne ke liye.
- Swagger: Browser mein API endpoints test karne ke liye.
- Unit testing: Service ya individual method ko independently test karna.
- Integration testing: Controller, database aur doosre components ke interaction ko test karna.

Tumhare Loan Application project mein, for example, Postman se foreclosure request API ko test kar sakte ho aur verify kar sakte ho ki valid request par 201 Created aur invalid request par appropriate error status mil raha hai.

3. Short Code Example

Example of a unit test using xUnit:

```
[Fact]
public void Add_ReturnsCorrectResult()
{
   int result = 10 + 20;
   Assert.Equal(30, result);
}
```

Here, [Fact] identifies a test method, and Assert.Equal() verifies the expected result.

4. Interview Tip

Remember the difference:

- Unit testing: Tests an individual unit of code.
- Integration testing: Tests interactions between components.
- Manual API testing: Uses tools such as Postman or Swagger.

Q21. Your API needs versioning (v1, v2). How do you design it?

→

“I would implement API versioning to introduce new features without breaking existing clients. For example, I would maintain /api/v1/loans for existing clients and /api/v2/loans for the updated response format. I would keep the old version compatible until clients migrate to the new version. For a larger application, I would use an API versioning library to manage versions systematically.”

Hinglish Explanation

Agar existing Angular frontend v1 use kar raha hai aur humein new features add karne hain, toh directly old API change nahi karenge. Hum v2 create karenge, jisse existing frontend break nahi hoga.

Code Example

```
[ApiController]

[Route("api/v1/loans")]

public class LoansV1Controller : ControllerBase

{

   [HttpGet]

   public IActionResult GetLoans()

   {

      return Ok(new { LoanId = 1, Amount = 50000 });

   }

}
```

The new controller can use the route api/v2/loans and return additional fields.

Interview tip: Mention backward compatibility and a planned deprecation strategy for old versions.

Q22. You need to return custom error messages in a consistent format for all API endpoints. How do you do it?

→

“I would implement centralized error handling using exception-handling middleware and return errors in a consistent format, preferably using the Problem Details standard. I would include an appropriate HTTP status code, a user-friendly message, and optionally a trace identifier. I would log detailed exception information on the server instead of exposing sensitive details to the client.”

Hinglish Explanation

Maan lo alag-alag APIs mein errors aa rahe hain. Agar har controller apna alag response return karega, toh frontend ko handle karna difficult hoga.

Isliye hum common error format maintain karenge, jaise:

{

```
  "status": 404,
  "title": "Loan not found",
  "traceId": "request-trace-id"
}
```

Code Example

ASP.NET Core provides Problem() for creating a Problem Details response:

```
[HttpGet("{id}")]
public IActionResult GetLoan(int id)
{
    var loan = FindLoan(id);

    if (loan == null)
        return Problem(statusCode: 404, title: "Loan not found");

    return Ok(loan);
}
```

Here, FindLoan() is a placeholder for your actual data-access logic. For consistency across all endpoints, centralize unexpected exception handling in middleware.

Interview tip: Differentiate expected errors, such as 404 Not Found, from unexpected server errors, such as 500 Internal Server Error.

Q23. An API is being called very frequently with the same data. How do you implement caching?

→

“I would identify whether the response contains frequently accessed data that does not change often. I would then use IMemoryCache for local caching or IDistributedCache, such as Redis, when multiple application instances need to share cached data. I would define an expiration policy and invalidate or update the cache when the underlying data changes. This reduces database load and improves response time.”

Hinglish Explanation

Suppose Loan Application mein interest rates baar-baar fetch ho rahe hain. Har request par database hit karne ke bajay, hum result ko cache mein rakh sakte hain.

Code Example

```
builder.Services.AddMemoryCache();

private readonly IMemoryCache _cache;
```

```
public LoansController(IMemoryCache cache)
{
    _cache = cache;
}


[HttpGet("interest-rates")]
public IActionResult GetInterestRates()
{
    var rates = _cache.GetOrCreate("InterestRates", entry =>
    {
        entry.AbsoluteExpirationRelativeToNow = TimeSpan.FromMinutes(10);
        return new[] { 8.5, 9.0, 9.5 };
    });


    return Ok(rates);
}
```

Interview tip: IMemoryCache stores data in application memory. IDistributedCache supports a shared cache across application instances. Also consider stale data and cache invalidation.

Q24. One of your APIs needs authentication with JWT. How do you implement it?

→

“I would configure JWT Bearer authentication in ASP.NET Core. After validating the user's credentials, the authentication endpoint would generate a signed JWT containing the required claims and an expiration time. The client would send the token in the Authorization header using the Bearer scheme. The API would validate the token and use the [Authorize] attribute to protect endpoints. I would also validate the issuer, audience, signing key, and token lifetime.”

Hinglish Explanation

JWT authentication ka flow hai:

1. User login karta hai.
2. Backend credentials verify karta hai.
3. Backend JWT token generate karta hai.
4. Angular token ko request header mein bhejta hai.
5. ASP.NET Core token validate karke access allow ya deny karta hai.

Code Example

Configure JWT authentication in Program.cs:

```
builder.Services.AddAuthentication(
   JwtBearerDefaults.AuthenticationScheme)
   .AddJwtBearer(options =>
   {
      options.TokenValidationParameters = new TokenValidationParameters
      {
         ValidateIssuer = true,
         ValidateAudience = true,
         ValidateIssuerSigningKey = true,
         ValidateLifetime = true,
         ValidIssuer = builder.Configuration["Jwt:Issuer"],
         ValidAudience = builder.Configuration["Jwt:Audience"],
         IssuerSigningKey = new SymmetricSecurityKey(
            Encoding.UTF8.GetBytes(
               builder.Configuration["Jwt:Key"]!))
      };
   });
```

Middleware order:

```
app.UseAuthentication();
app.UseAuthorization();
app.MapControllers();
```

Protect an endpoint:

```
[Authorize]
[HttpGet("profile")]
public IActionResult GetProfile()
{
   return Ok("Access granted");
}
```

This assumes the required JWT configuration values and namespaces are present.

Q25. You need to call another API inside your Web API (chained calls). How do you design this?

→

“I would use IHttpClientFactory to communicate with the external API, preferably through a typed HttpClient registered using dependency injection. I would keep the external API communication in a dedicated service instead of putting it directly in the controller. I would use asynchronous methods, configure timeouts, handle unsuccessful responses, and apply appropriate retry policies for transient failures. If the external API is unavailable, I would return a suitable error or fallback response.”

Hinglish Explanation

Maan lo Loan Application API ko loan eligibility calculate karne se pehle external CIBIL API call karni hai.

Flow:

Controller → Loan Service → CIBIL API → Response → Eligibility result

External API call ko dedicated service mein rakhne se code maintainable aur testable rehta hai.

Code Example

Register a named client:

```
builder.Services.AddHttpClient("CibilApi", client =>
{
   client.BaseAddress = new Uri("https://example.com/");
   client.Timeout = TimeSpan.FromSeconds(10);
});
```

Use it inside a service:

```
public class CibilService
{
   private readonly HttpClient _client;

   public CibilService(IHttpClientFactory factory)
   {
      _client = factory.CreateClient("CibilApi");
   }

   public async Task<string> GetScoreAsync(
      CancellationToken cancellationToken)
   {
```

```
    var response = await _client.GetAsync(
        "api/score",
        cancellationToken);

    response.EnsureSuccessStatusCode();

    return await response.Content.ReadAsStringAsync(
        cancellationToken);
}
```

Q26. How will you handle exceptions globally in your Web API?

→

“I would implement global exception handling middleware to handle unexpected exceptions centrally instead of writing repetitive try-catch blocks in every controller. The middleware would log the exception, return an appropriate HTTP status code, and provide a consistent error response. I would use the built-in UseExceptionHandler() or custom middleware and ensure that sensitive internal details are not exposed to clients.”

2. Hinglish Explanation

Agar Loan Application API mein database error aa jata hai, toh har controller mein try-catch likhne ke bajay global middleware exception handle karega.

- Exception ko log karega.
- Appropriate status code return karega.
- Client ko safe error message dega.

3. Code Example

In Program.cs:

app.UseExceptionHandler();

This uses ASP.NET Core's built-in exception handler. Configure it with appropriate error handling services or an error-handling endpoint according to your application requirements.

4. Interview Tip

Global exception middleware handles unexpected errors centrally. Expected conditions, such as a loan not being found, should normally return an appropriate response such as 404 Not Found.

Q27. A client complains that your API is slow because of returning too much data. How do you optimize it?

→

“I would optimize the API by returning only the fields the client needs, implementing pagination, and avoiding unnecessary database queries. In Entity Framework Core, I would use projection with Select(), AsNoTracking() for read-only queries, and appropriate database indexes. I would also measure query execution time and response size to identify the actual bottleneck.”

2. Hinglish Explanation

Suppose API ek saath 1 lakh employee records return kar rahi hai. Isse database, server aur network par load badhega.

Main:

- Pagination use karunga.
- Sirf required columns fetch karunga.
- Read-only queries mein AsNoTracking() use karunga.
- Database queries aur indexes analyze karunga.

3. Code Example

```
[HttpGet]
public async Task<IActionResult> GetEmployees(
   int pageNumber = 1, int pageSize = 20)
{
   if (pageNumber < 1 || pageSize < 1 || pageSize > 100)
      return BadRequest("Invalid pagination parameters.");

   var employees = await _db.Employees
      .AsNoTracking()
      .OrderBy(e => e.EmployeeId)
      .Select(e => new
      {
         e.EmployeeId,
         e.Name,
         e.Email
      })
      .Skip((pageNumber - 1) * pageSize)
```

```
        .Take(pageSize)
        .ToListAsync();

    return Ok(employees);
}
```

Q28. How do you handle transactions if you are saving data into multiple tables?

→

“When an operation modifies multiple database tables, I use a database transaction to ensure data consistency. A transaction follows the ACID principles. If all operations succeed, I commit the transaction; if an operation fails, I roll it back. With Entity Framework Core, I can use BeginTransactionAsync(), CommitAsync(), and RollbackAsync() when an explicit transaction is required.”

2. Hinglish Explanation

Maan lo loan application submit karte waqt humein LoanAccounts aur EmiSchedules dono tables mein data save karna hai.

Agar loan account save ho gaya, lekin EMI schedule save nahi hua, toh data inconsistent ho sakta hai. Transaction ensure karta hai ki dono operations successful hon, ya failure hone par changes rollback ho jayein.

3. Code Example

```
await using var transaction =
    await _db.Database.BeginTransactionAsync();

try
{
    _db.LoanAccounts.Add(loan);
    await _db.SaveChangesAsync();

    _db.EmiSchedules.AddRange(emiSchedules);
    await _db.SaveChangesAsync();

    await transaction.CommitAsync();
}
```

```
catch
{
    await transaction.RollbackAsync();
    throw;
}
```

Here, loan and emiSchedules represent prepared entity objects. In many cases, a single SaveChangesAsync() is already transactional; an explicit transaction is useful when multiple database operations must succeed or fail together.

4. Interview Tip

Know the four ACID properties:

- Atomicity: All operations succeed or none do.
- Consistency: Data remains valid.
- Isolation: Concurrent transactions are controlled.
- Durability: Committed changes persist.

Q29. You need to secure an API for internal apps only. How do you restrict external users?

→

"I would first clarify what 'internal only' means for the application. I would use authentication and authorization, and restrict network access through a private network, firewall, VPN, or API gateway where appropriate. For service-to-service communication, I would use secure credentials or OAuth2 client credentials, and potentially mutual TLS. I would not rely on CORS alone because CORS is a browser security mechanism, not an API access-control mechanism."

2. Hinglish Explanation

Agar API sirf company ke internal applications ke liye hai, toh sirf CORS configure karna enough nahi hai.

Main:

- Authentication enforce karunga.
- Service identity ya client credentials validate karunga.
- Zarurat ke according API ko private network ya firewall se restrict karunga.
- HTTPS use karunga.

3. Code Example

Require authentication for an endpoint:

```
[Authorize]

[HttpGet("internal/reports")]

public IActionResult GetInternalReports()

{

   return Ok("Internal report data");

}
```

[Authorize] ensures that the caller must be authenticated according to the configured authentication scheme. Network restrictions and service identity checks must be configured separately.

4. Interview Tip

CORS is not authentication. Non-browser clients can call an API even when browser CORS restrictions would block a frontend request.

Q30. You have two roles (Admin, User). Admin can access everything; User can only read. How do you implement this?

→

“I would implement role-based authorization using the [Authorize] attribute. I would allow both Admin and User roles to access read-only endpoints, while restricting create, update, and delete operations to the Admin role. The user's role would be established through trusted authentication claims, such as claims in a validated JWT. I would enforce these restrictions on the server, not just in the frontend.”

2. Hinglish Explanation

- Admin: Read, create, update aur delete kar sakta hai.
- User: Sirf read operations kar sakta hai.

Frontend par buttons hide karna enough nahi hai. Backend endpoints par bhi authorization lagana zaroori hai.

3. Code Example

Read endpoint — both roles:

```
[Authorize(Roles = "Admin,User")]

[HttpGet]

public IActionResult GetLoans()

{

   return Ok("Loan list");

}
```

Create endpoint — Admin only:

```
[Authorize(Roles = "Admin")]
[HttpPost]
public IActionResult CreateLoan()
{
   return Ok("Loan created");
}
```

Apply the same Admin-only restriction to update and delete endpoints. Ensure the JWT contains a role claim recognized by ASP.NET Core's role-based authorization.

4. Interview Tip

- [Authorize] — requires authentication.
- [Authorize(Roles = "Admin")] — restricts access to Admin.
- [Authorize(Roles = "Admin,User")] — permits either role.

Q31. How do you prevent CSRF attacks in ASP.NET Core MVC?

→

“CSRF stands for Cross-Site Request Forgery. It is an attack in which an attacker tricks an authenticated user's browser into sending an unwanted request to a website.

In ASP.NET Core MVC, I prevent CSRF attacks by using anti-forgery tokens. The server generates a token, and the application includes it in the form submission. The server validates the token before processing the request. I use the [ValidateAntiForgeryToken] attribute on sensitive actions and can use [AutoValidateAntiforgeryToken] to apply validation more broadly to unsafe HTTP methods.”

2. Hinglish Explanation

CSRF attack mein attacker user ke logged-in browser ka misuse karke unwanted request bhejne ki koshish karta hai.

For example, user banking website mein logged in hai. Attacker usse malicious website par le jaata hai, jo user ke browser se unwanted request trigger karne ki koshish karti hai.

Anti-forgery token verify karta hai ki request mein valid token hai ya nahi.

3. Code Example

Razor form:

```
<form asp-action="Transfer" method="post">
```

```
    @Html.AntiForgeryToken()
    <input name="amount" type="number" />
    <button type="submit">Transfer</button>
</form>
```

Controller:

```
[HttpPost]
[ValidateAntiForgeryToken]
public IActionResult Transfer(decimal amount)
{
    return Ok("Request validated");
}
```

4. Interview Tip

CSRF protection is especially important for cookie-based authentication because browsers automatically send cookies with matching requests. For bearer tokens explicitly added by JavaScript, the CSRF risk is different, though XSS and token storage still need careful handling.

Q32. How will you protect your Web API endpoints from brute-force attacks?

→

"A brute-force attack occurs when an attacker repeatedly tries different passwords or credentials to gain unauthorized access. I would protect the API using rate limiting, login attempt restrictions, temporary account lockouts, and monitoring. I would also use multi-factor authentication where appropriate, enforce HTTPS, and log repeated failed attempts. For larger systems, I would consider additional protection at the API gateway or web application firewall level."

2. Hinglish Explanation

Suppose koi attacker login API par thousands of passwords try kar raha hai. Isko brute-force attack kehte hain.

Protection ke liye:

- Ek client ke requests limit karenge.
- Repeated failed logins par temporary lockout laga sakte hain.
- Failed attempts log aur monitor karenge.
- MFA use kar sakte hain.

3. Code Example

In ASP.NET Core, rate limiting can be configured with a fixed-window policy:

```
builder.Services.AddRateLimiter(options =>
{
    options.AddFixedWindowLimiter("LoginPolicy", limiter =>
    {
        limiter.PermitLimit = 5;
        limiter.Window = TimeSpan.FromMinutes(1);
        limiter.QueueLimit = 0;
    });
});
```

Register the middleware:

```
app.UseRateLimiter();
```

Apply the policy:

```
[EnableRateLimiting("LoginPolicy")]
[HttpPost("login")]
public IActionResult Login()
{
    return Ok();
}
```

This is a basic rate limit example, not a complete login security solution. In production, also consider account-based throttling and distributed rate limiting when multiple API instances are running.

4. Interview Tip

Rate limiting restricts request frequency. Account lockout temporarily blocks repeated failed login attempts. Both can help, but neither should be implemented in a way that makes it easy to deny service to legitimate users.

Q33. You need to log which user accessed which endpoint. How will you implement it?

→

“I would implement centralized request logging using ASP.NET Core logging middleware or an established logging framework. For authenticated requests, I would capture the user identifier from the authenticated claims, along with the HTTP method, endpoint, timestamp, response status code, and correlation ID. I would send these logs to a centralized logging system when needed. I would avoid logging passwords, access tokens, or other sensitive information.”

2. Hinglish Explanation

Maan lo Loan Application project mein humein track karna hai ki kis user ne foreclosure API access ki aur response status kya tha.

Hum log karenge:

- User ID
- Endpoint aur HTTP method
- Request ka time
- Response status code
- Correlation ID

3. Code Example

A simple middleware:

```
app.Use(async (context, next) =>
{
    await next();

    var userId = context.User.FindFirst(
        System.Security.Claims.ClaimTypes.NameIdentifier)?.Value;

    var logger = context.RequestServices
        .GetRequiredService<ILogger<Program>>();

    logger.LogInformation(
        "User {UserId} accessed {Method} {Path}. Status: {StatusCode}",
        userId,
        context.Request.Method,
        context.Request.Path,
```

```
    context.Response.StatusCode);
});
```

Place this middleware after authentication so authenticated user claims are available. In production, consider structured audit logging and ensure the log sink is protected.

4. Interview Tip

Application logs help with debugging. Audit logs are specifically intended to record security-relevant actions and should have appropriate access controls and retention policies.

Q34. Your app works locally but fails in production with connection string issues. How will you troubleshoot?

→

"I would first inspect the production error logs and verify that the application is loading the correct configuration. I would check the database server address, database name, credentials, network connectivity, firewall rules, and database permissions. I would also confirm that production environment variables or secret stores are correctly configured and that the application is not accidentally using development settings. I would avoid exposing connection strings or credentials in logs."

2. Hinglish Explanation

Local machine par API chal rahi hai, lekin production mein database connect nahi ho raha. Main step-by-step check karunga:

1. Production connection string correct hai ya nahi.
2. Database server reachable hai ya nahi.
3. Username/password aur permissions correct hain ya nahi.
4. Firewall aur network access allow hai ya nahi.
5. Application correct environment configuration load kar rahi hai ya nahi.

3. Code Example

Read the connection string from configuration:

```
var connectionString = builder.Configuration.GetConnectionString("DefaultConnection");

builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(connectionString));
```

For production, provide the DefaultConnection value through a secure configuration provider, such as an environment variable or secret manager, rather than committing credentials to source control.

4. Interview Tip

A connection string can be syntactically correct but still fail because of network access, SQL Server configuration, credentials, or permissions. Diagnose the actual exception instead of guessing.

Q35. You deployed your API, but Angular frontend gets CORS errors. How do you fix it?

→

“I would inspect the browser's developer console and network tab to identify the blocked origin and whether the preflight request is failing. Then I would configure a CORS policy in ASP.NET Core to allow the exact production Angular origin, required HTTP methods, and headers. I would ensure CORS middleware runs in the correct order and check whether credentials are required. I would not use CORS as a replacement for authentication or authorization.”

2. Hinglish Explanation

Local par Angular localhost:4200 se API call kar raha tha, lekin production mein frontend aur backend ke domains different hain.

Main:

- Browser console mein exact CORS error check karunga.
- Production Angular origin allow karunga.
- Required headers aur methods allow karunga.
- Preflight OPTIONS request aur middleware order check karunga.

3. Code Example

Program.cs:

```
builder.Services.AddCors(options =>
{
   options.AddPolicy("FrontendPolicy", policy =>
   {
      policy.WithOrigins("https://your-frontend.example.com")
          .AllowAnyHeader()
          .AllowAnyMethod();
   });
});
```

Configure the middleware:

```
app.UseRouting();
app.UseCors("FrontendPolicy");
app.UseAuthentication();
app.UseAuthorization();
app.MapControllers();
```

Replace the example origin with your actual production frontend origin. If cookies or other browser credentials are required, configure credentials deliberately and never combine AllowAnyOrigin() with AllowCredentials().

4. Interview Tip

CORS is enforced by browsers. A server-to-server request may work even when the same request from Angular is blocked by the browser. Always check the exact origin and preflight response.

You've now covered five more scenario-based questions (Q31–Q35) from your interview preparation sequence.